ARVYON · INFORME DE INVESTIGACIÓN

Fábricas de IA, arneses y enrutamiento de modelos

Qué plantea el video de Eric Risco sobre las *software factories*, qué le faltó, qué son el *harness engineering* con Pi y el enrutamiento de modelos, y qué de todo esto debes aprender como consultora de IA.

Fecha	2026-11-11
Versión	1 · Borrador para tu revisión. **Todavía no está en el cerebro.**
Pregunta de fondo	¿Estos temas aportan a mi aprendizaje y a mi oficio de consultor de IA para pymes colombianas?
Fuente principal	«Programar con agentes ya no basta: monta tu Software Factory», Eric Risco, YouTube, 2026-10-23 (17 min 21 s). Transcripción completa leída.
Fuentes de contraste	Stripe, StrongDM, OpenAI (vía InfoQ), Thoughtworks en martinfowler.com, FinOps Foundation, Anthropic, sitio y blog de Pi, y dos artículos académicos. Lista completa en el capítulo 11.
Fase del proyecto	Fase 1 · Aprender (hasta el 2026-12-15). Este informe no propone ninguna acción con terceros.

Contenido

1 Respuesta corta

2 El video, paso a paso

3 Evaluación: aciertos, exageraciones y errores

4 Lo que le faltó: la fábrica completa

5 La fábrica de contenido

6 *Harness engineering* y Pi

7 Enrutamiento de modelos: la disciplina que buscabas

8 Temas que faltaban en tu lista

9 Qué significa para Arvyon

10 Glosario

11 Fuentes y límites de esta investigación

Cómo leer este informe

Uso los mismos recuadros que la wiki del cerebro, para que el paso al cerebro sea directo cuando lo apruebes.

HECHO
Dato que comprobé en una fuente que cito.

ESTIMACIÓN
Cálculo mío, con la fórmula a la vista.

POR VERIFICAR
Algo que no pude confirmar en una fuente primaria.

ERROR O INCOHERENCIA
Algo del video que no cuadra, con la razón.

PARA ARVYON
Lo que el punto significa para tu estudio o para la consultora.

Los términos en inglés aparecen la primera vez con su traducción. El glosario del capítulo 10 los reúne todos.

1 · Respuesta corta

Los cuatro temas valen la pena como **conceptos**, y los cuatro son en el fondo el mismo: cómo organizar el trabajo de una IA para que sea repetible, comprobable y barato. Ninguno te pide construir nada ahora ni aprender a programar antes del 15 de diciembre.

Tema	¿Te aporta?	¿Cuándo estudiarlo?
1 · Fábrica de software con IA	**Sí, el concepto** Una «fábrica» es un proceso repetible hecho de tres tipos de pasos: código, agente y aprobación humana. Es exactamente lo que vas a diseñar para una pyme (cotizaciones, atención por WhatsApp, reportes), solo que el video lo aplica a programar. **No, el proyecto** Montar una fábrica de código exige saber programar y no es lo que compra tu cliente.	La idea, ya. La práctica, en las semanas 3 y 4 (bloques 4 y 5, con n8n).
2 · Fábrica de contenido	**Sí, para YouTube** Es el mismo patrón aplicado a editar video. Sirve a tu canal, pero solo cubre la posproducción: la idea, el guion y la cámara siguen siendo tuyos.	Fase 2 (bloque 12). Hoy basta con el concepto.
3 · *Harness engineering* y Pi	**Sí, el concepto** El arnés es todo lo que rodea al modelo: instrucciones, memoria, herramientas y comprobaciones. Ya usas uno: el archivo `CLAUDE.md` y el revisor de la wiki son un arnés. **Pi, todavía no** Pi es una herramienta de terminal para programadores.	El concepto, en la semana 2. Pi, opcional, después del 15 de diciembre.
4 · La disciplina para elegir modelo	**Sí** Se llama **enrutamiento de modelos** (*model routing*) y forma parte de **FinOps para IA**, la gestión del gasto en IA. Toda pyme te preguntará cuánto cuesta, y esta es la respuesta ordenada.	Semana 2 (bloque 3, herramientas) y semana 5 (bloque 10, retorno de la inversión).
5 · Lo que faltaba	Ocho temas (capítulo 8). Los tres que más pesan: **evaluaciones** (cómo saber si la IA lo hace bien), **seguridad de agentes** (inyección de instrucciones) y **flujo o agente** (cuándo no hace falta un agente).	Repartidos entre las semanas 3 y 5 y la Fase 2.

Veredicto sobre el video

Es un **buen mapa con un relato optimista**. La estructura que enseña es correcta y coincide con lo que publican Stripe, OpenAI y Thoughtworks. Lo que no cuenta es lo que hace que una fábrica funcione de verdad: cuánto cuesta, cómo se protege de entradas maliciosas, qué comprobaciones automáticas la sostienen, cuántas vueltas puede dar antes de parar y qué datos demuestran que funciona. Además, la herramienta que anuncia (RSC Factory) no estaba publicada el 2026-11-11.

> **PARA ARVYON**
>
> La idea más valiosa del video no es técnica. Es esta: **un proceso sirve para la IA si es repetible y si cada paso deja rastro**. Esa frase es un criterio de diagnóstico que puedes aplicar a cualquier empresa, y por eso te propongo guardarla en el cerebro como concepto propio.

2 · El video, paso a paso

Dato	Valor
Título	«Programar con agentes ya no basta: monta tu Software Factory»
Canal y autor	Eric Risco | Inteligencia Artificial. Según su perfil de GitHub, es *AI Tech Lead* en Andorra Telecom.
Publicación y duración	2026-10-23 · 17 min 21 s · 10 capítulos
Alcance	424.685 vistas y 8.167 «me gusta» el 2026-11-11 (19 días después)
Qué vende	Su proyecto de código abierto RSC Harness (586 estrellas en GitHub el 2026-11-11) y el anuncio de RSC Factory, aún no publicado.

Método: descargué los subtítulos automáticos de YouTube y los leí completos. Los subtítulos automáticos tienen errores («Cloud Code» es Claude Code, «Sonet» es Sonnet). No vi la imagen del video: el diagrama lo reconstruí con lo que el autor dice y con un repositorio de un tercero que lo capturó.

00:00 **El problema.** Chatear con un agente de programación ya no basta, porque todo pasa por ti: «el cuello de botella eres tú», aunque abras cuatro ventanas. Afirma que en Estados Unidos «toda la ingeniería de software» ya trabaja con *software factories*.

01:06 **Hay trabajo que ya no te necesita.** Errores de producción, pruebas y desarrollos pequeños se pueden delegar. La idea: el trabajo entra por un lado, sale por otro y en medio pasan cosas sin ti, pero el sistema lo diseñaste tú.

01:38 **Qué es una fábrica de IA.** «Cualquier proceso que sea repetible y auditable con inteligencia artificial dentro.» Usa la analogía de una fábrica de coches: cada máquina recibe algo, entrega algo y su resultado se puede revisar, porque un error se arrastra a las máquinas siguientes. Distingue **tres tipos de pasos**: de código (siempre igual), de agente (decide según lo que entra) y de aprobación humana. A cada paso hay que darle cuatro cosas: qué hace, cómo lo hace, cuál es el resultado y cómo se revisa.

04:00 **El *brain* (la memoria).** Sin contexto continuo el agente se pierde. Ejemplo: decidiste usar PostgreSQL hace tres meses, el agente lo olvida y te propone MySQL. La solución es una memoria persistente donde se registra cada decisión y que el agente consulta antes de actuar.

05:30 **Su fábrica: entrada, especificación y construcción.** Por la misma puerta entran errores de producción (detectados con Sentry), *issues* de GitHub y tareas suyas. Un agente *spec writer* (redactor de la especificación), con el modelo Opus, consulta la memoria y escribe la especificación. Un agente *builder* (constructor), también Opus, crea una rama y programa escribiendo primero la prueba y después el código (TDD).

08:12 **Los cuatro revisores.** El constructor lanza cuatro subagentes con contexto limpio: seguridad (Sol, de OpenAI), experiencia de uso o UX (Sol), interfaz o UI (Astra, de OpenAI) y código (Sonnet). Usa otros modelos porque el que construyó «se sesgaría» al revisarse a sí mismo.

10:14 **El aprobador y la puerta de riesgo.** Un agente aprobador (Opus) reúne los cuatro veredictos. Si alguno es negativo, devuelve el trabajo al constructor, y así hasta que los cuatro pasan. Después, otro agente decide si el cambio es lo bastante grande para pedir revisión humana: cambiar el color de un botón sube directo; tocar la base de datos o el núcleo del negocio le llega como un *pull request* para aprobar a mano. Reconoce que aquí «no basta solo con un agente»: hay que poner código que bloquee.

12:32 **En paralelo y para quien no programa.** Cada tarea tiene su identificador y su rama, así que se pueden lanzar muchas a la vez. Propone conectar la fábrica a Slack para que marketing y ventas pidan cambios (páginas, frases, el aviso legal) sin pasar por un desarrollador.

13:51 **La fábrica de contenido.** El mismo patrón edita sus videos: le sube la grabación en bruto y la fábrica recorta silencios y errores, añade animaciones y transiciones, une todo y crea las miniaturas. Él revisa y sube el video a mano.

15:49 **Por qué ahora.** Cita a ingenieros de OpenAI (más de un millón de líneas sin escribir ninguna a mano) y a Stripe (más de mil *pull requests* por semana hechos por agentes). Regla que propone: «si es algo que haces más de una vez a la semana, tiene todo el sentido». Anuncia RSC Factory y pide apoyo al video.

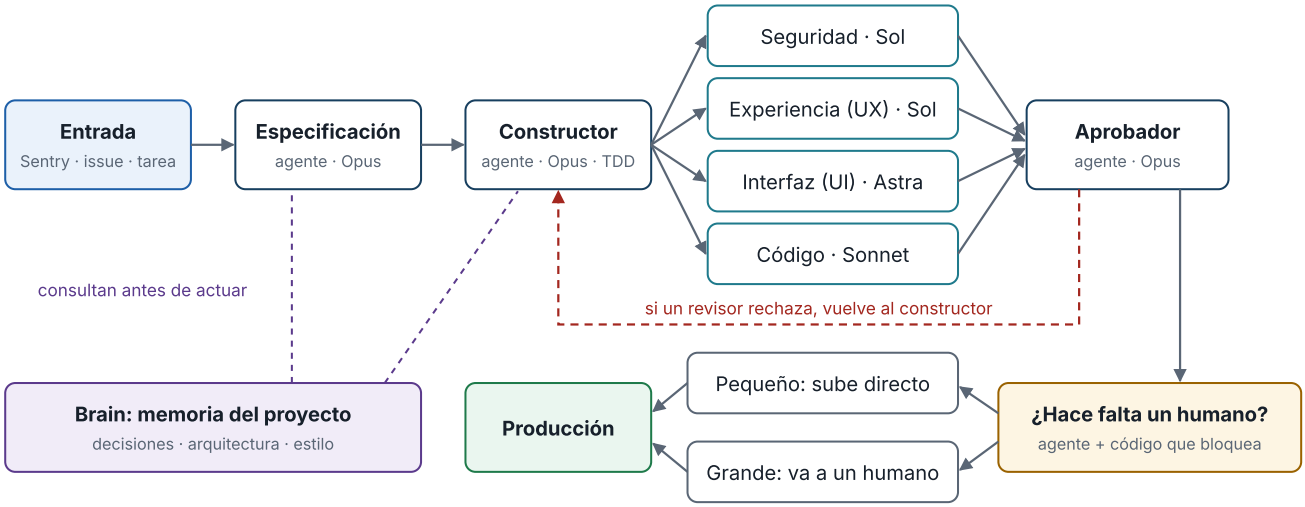


Figura 1. La fábrica de software del video. Reconstrucción propia a partir de la transcripción (minutos 05:30 a 12:32).

3 · Evaluación: aciertos, exageraciones y errores

3.1 Lo que el video plantea bien

- **Los tres tipos de pasos.** Mezclar pasos de código, pasos de agente y aprobación humana es justo lo que describe Stripe: su sistema intercala bucles de agente con código determinista (git, comprobadores de estilo, pruebas) para que los pasos obligatorios se ejecuten siempre, decida lo que decida el modelo.
- **La especificación como contrato.** El constructor no interpreta la petición: ejecuta un documento que ya dice qué se acepta. Es la idea central del desarrollo guiado por especificaciones.
- **La memoria.** El equipo de OpenAI llegó a lo mismo: un archivo corto de instrucciones que funciona como mapa y una carpeta de documentación como fuente única de verdad para los agentes.
- **Revisar con un modelo distinto.** Tiene base: un estudio de NeurIPS 2024 mostró que los modelos reconocen sus propios textos y los puntúan mejor que los ajenos.
- **Un agente no basta para la puerta de riesgo.** Lo dice él mismo y es correcto: hacen falta reglas en código.
- **Cada puesto paga el modelo que necesita.** Es enrutamiento de modelos (capítulo 7), aunque no lo nombra.

3.2 Afirmaciones contrastadas

Lo que dice el video	Veredicto	Evidencia
«En Estados Unidos, toda la ingeniería de software ya ha pasado al siguiente nivel.»	Exagerado	Los casos publicados son de equipos de vanguardia. El informe DORA 2025 (casi 5.000 encuestados) concluye que la IA es un «amplificador»: sube la velocidad de entrega, pero también la inestabilidad. El ensayo de METR de 2025 midió que desarrolladores expertos tardaron un 19 % más con IA; su actualización de febrero de 2026 no fue concluyente.
Ingenieros de OpenAI crearon una solución de más de un millón de líneas sin escribir ninguna a mano.	Cierto, con matices	Es el artículo «Harness engineering» de OpenAI (febrero de 2026): cinco meses, una beta interna, cerca de un millón de líneas. No fue «el otro día», y no era un producto abierto al público.
Stripe hizo más de 1.000 *pull requests* en una semana con agentes.	Cierto, con un matiz clave	Stripe, 2026-02-09: más de mil cambios integrados por semana, sin código humano, pero **revisados por personas**. Se apoyan en más de tres millones de pruebas, en máquinas aisladas de producción y de internet, y en un máximo de dos rondas de comprobación.
Sol es de OpenAI y Astra está «muy enfocado» al uso del computador.	Cierto en lo esencial	OpenAI presenta a Astra (GPT-6, septiembre de 2026) como su modelo más capaz en uso del computador. Que esté «entrenado con refuerzo para revisar la interfaz» no lo pude confirmar.
Otro modelo evita que el constructor se sesgue al revisarse.	Plausible	El estudio de NeurIPS 2024 midió el sesgo en resúmenes de texto, no en código. La idea es razonable; la prueba directa no existe.

«Cualquier error [...] se soluciona solo.»	Anécdota	No da ninguna cifra (tasa de éxito, costo, regresiones) y el software donde lo usa «lo voy a lanzar muy pronto», es decir, aún no tiene usuarios reales a escala.
«Puedes sacar el trabajo de todo un equipo.»	Sin respaldo	En OpenAI el equipo creció de tres a siete ingenieros durante el experimento (según resúmenes del artículo): los agentes no sustituyeron al equipo, cambiaron su trabajo.
«Si es algo que haces más de una vez a la semana, tiene todo el sentido.»	Regla incompleta	Falta comparar lo que cuesta montar y mantener la fábrica con lo que ahorra. Anthropic recomienda lo contrario como punto de partida: la solución más simple posible, y complejidad solo cuando haga falta.
Se pueden llevar aplicaciones a producción «sin saber programar».	Riesgoso	La puerta humana solo protege si la persona puede juzgar lo que aprueba. Quien no lee código aprueba a ciegas justo los cambios grandes.
Opus 5.5 «edita» los videos.	Impreciso	El modelo razona y da órdenes a programas de edición; no recorta el video él mismo (capítulo 5).

3.3 Errores e incoherencias internas

1 · EL CONSTRUCTOR LANZA A SUS PROPIOS REVISORES

Dice que «el agente *builder* lanza cuatro subagentes». Quien es revisado no debería controlar su revisión: puede resumir su trabajo a su favor o saltarse a un revisor. Lo correcto es que los lance un paso de código, fuera del alcance del constructor. Una implementación de terceros basada en el video ya lo corrige con un guion (`launch-reviewers.sh`).

2 · EL APROBADOR ES EL MISMO MODELO QUE CONSTRUYÓ

Justifica usar Sol y Astra para que Opus no se revise a sí mismo, pero el veredicto final lo consolida Opus. El sesgo que quiso evitar vuelve a entrar en el último paso.

3 · LA REVISIÓN DE CÓDIGO «NO ME PREOCUPA MUCHO»

Entrega la revisión de código a Sonnet, de la misma casa que Opus (Anthropic), y a un modelo menor. Contradice su propio argumento de las «familias distintas», y la corrección del código es precisamente lo que decide si el cambio rompe algo.

4 · EL AVISO LEGAL COMO TAREA PEQUEÑA

Pone de ejemplo que ventas o marketing añadan «algo en el aviso legal» y que la fábrica decida si necesita humano. Su agente mide el riesgo por lo que el cambio toca en el sistema (base de datos, núcleo). Un cambio en un texto legal es diminuto en código y grande en consecuencias. **El tamaño del cambio no es su riesgo.**

5 · LAPSUS MENORES

Dice dos veces que la experiencia de uso la revisa Sol. Dice «pasa directamente a la PR y se sube directamente a main», cuando la ruta rápida es justo la que no espera a nadie. Llama «audio en bruto» a la grabación de video. Y usa «AI Factory» en un sentido distinto del habitual: Nvidia llama así a los centros de datos que producen *tokens*. Si buscas el término, encontrarás las dos cosas mezcladas.

4 · Lo que le faltó: la fábrica completa

El video enseña la línea de montaje. Estas doce piezas son las que, según las empresas que sí publican resultados, hacen que la línea no produzca errores en serie.

#	Pieza que falta	Por qué importa y qué evidencia hay
1	**Comprobaciones automáticas que no dependan de una IA**	Los cuatro revisores del video son modelos opinando. Thoughtworks distingue dos clases de control: el **computacional** (pruebas, comprobadores de estilo y de tipos: rápido, barato, siempre igual) y el **inferencial** (una IA que juzga: lento, caro, variable). El video solo tiene del segundo. Stripe se apoya primero en el primero.
2	**Pruebas que el constructor no pueda acomodar**	El mismo agente escribe la prueba y el código, así que puede escribir una prueba fácil. StrongDM guarda sus «escenarios» fuera del código, como un examen que el alumno no ve antes. Thoughtworks dice de las pruebas generadas por IA: «todavía no es suficiente».
3	**Límite de vueltas y de gasto**	El bucle «hasta que el aprobador decida» no tiene tope. Stripe corta en dos rondas de comprobación, por costo en *tokens*, cómputo y tiempo.
4	**El costo**	No se menciona en 17 minutos. Ver la estimación de abajo.
5	**Defensa de la entrada**	Errores de producción, *issues* y mensajes de Slack son texto escrito por otros, y un agente puede obedecerlo. En febrero de 2026, el título de un *issue* de GitHub manipuló al robot de clasificación del proyecto Cline y terminó en la publicación no autorizada de una versión que se instaló en unas 4.000 máquinas («Clinejection»).
6	**Permisos mínimos y entorno aislado**	Los agentes de Stripe trabajan en máquinas aisladas de producción y de internet. El video no dice dónde corre su fábrica ni qué claves tiene a mano.
7	**Reglas de riesgo escritas, no opinadas**	La ruta debería decidirla una tabla fija (qué archivos toca, si hay dinero, datos personales o textos legales) y, ante la duda, ir al humano. Un agente puede aportar, no decidir solo.
8	**Marcha atrás y vigilancia después de publicar**	¿Qué pasa si el arreglo automático rompe otra cosa? Hace falta poder deshacer rápido y vigilar. DORA 2025 advierte que más cambios sin buenos controles traen más inestabilidad.
9	**Medición: ¿la fábrica funciona?**	Sin un conjunto de casos de prueba y sin números (porcentaje de tareas bien resueltas, costo por resultado correcto) no se sabe si mejora o empeora. Es el tema de las evaluaciones (capítulo 8).
10	**Quién cuida la memoria**	Una memoria desactualizada o contradictoria engaña al agente con más autoridad que no tener ninguna. Con tareas en paralelo, además, varias escriben a la vez. Alguien debe podarla.
11	**Un humano que pueda juzgar**	Aprobar veinte cambios al día cansa, y la gente termina diciendo sí a todo. La puerta humana necesita pocas aprobaciones, bien explicadas, y una persona capaz de entenderlas.
12	**Cuándo no montar una fábrica**	Si el proceso cambia cada vez, tiene poco volumen o un error cuesta muy caro, automatizarlo entero no conviene.

ESTIMACIÓN · CUÁNTAS LLAMADAS A MODELOS HACE UNA SOLA TAREA

Una pasada limpia son 8 llamadas: especificación (1) + construcción (1) + revisores (4) + aprobador (1) + agente de riesgo (1). Cada rechazo añade al menos 6 más (construcción, 4 revisores, aprobador). Con dos rechazos: 8 + 2 × 6 = **20 llamadas**, cinco de ellas con el modelo más caro y todas con contextos largos. Sin precios ni tamaños reales no puedo pasar esto a dinero, pero explica por qué las empresas ponen topes.

HECHO · EL ARNÉS TAMBIÉN CUESTA

Naresh Jain midió en octubre de 2026 (martinfowler.com) una misma tarea con y sin arnés de comprobaciones. Con el modelo potente (Sol), el arnés gastó unas 10,5 veces más *tokens*, tardó casi 3 veces más y costó unas 7 veces más (2,68 frente a 0,41 dólares), con el mismo resultado: 10 de 10 aceptadas. Con el modelo barato (Luna), el arnés subió las aceptadas de 3 a 10 de 10 y aun así costó 0,27 dólares, un 33 % menos que Sol sin arnés. Límites que el propio autor señala: una sola tarea, en Java, con muestra pequeña.

Cuatro fábricas reales, comparadas

	OpenAI (feb 2026)	Stripe (feb 2026)	StrongDM (feb 2026)	Video de Risco (oct 2026)
¿Quién escribe?	Agentes	Agentes	Agentes	Agentes
¿Quién revisa?	Agentes y personas	Personas, siempre	Nadie lee el código	Cuatro agentes; persona solo en cambios grandes
¿En qué se apoya?	Documentación como fuente de verdad y reglas de arquitectura automáticas	Más de 3 millones de pruebas, máquinas aisladas, tope de 2 rondas	Escenarios fuera del código y copias de los servicios externos para probar	Memoria, prueba primero y revisores con otros modelos
Escala publicada	Cerca de 1 millón de líneas en 5 meses	Más de 1.000 cambios por semana	Equipo de 3 personas; no da volumen	Ninguna cifra

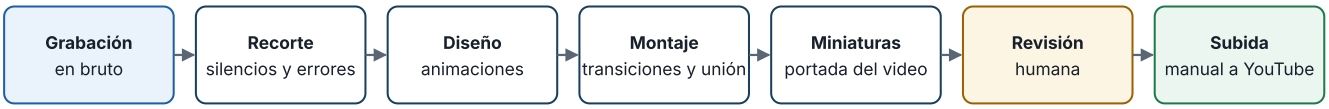
StrongDM es el caso extremo, llamado a veces «fábrica oscura» (sin luces, porque no hay nadie mirando). Su guía dice que una fábrica así debería gastar al menos 1.000 dólares diarios en *tokens* por ingeniero. Es una referencia aspiracional suya, no un gasto medido.

PARA ARVYON

Las doce piezas son una **lista de comprobación de consultor**. Cuando una pyme te diga «quiero automatizar esto con IA», estas son las preguntas que separan una propuesta seria de un demo: ¿cómo sabemos que lo hizo bien?, ¿cuánto cuesta cada caso?, ¿quién aprueba qué?, ¿qué pasa si se equivoca?, ¿cómo lo deshacemos?

5 · La fábrica de contenido

Es el segundo tema que te interesó. En el video ocupa dos minutos (13:51 a 15:49) y el autor promete un video aparte si se lo piden. Aquí lo completo.



Lo que queda fuera de la fábrica: la idea, el guion, la persona frente a la cámara y el criterio de qué publicar.

Figura 2. La fábrica de contenido que describe el video. Reconstrucción propia.

Cómo funciona de verdad

El modelo no «ve» ni recorta el video como lo haría un editor. Hace de director: lee la transcripción con sus tiempos, decide qué cortar y da órdenes a programas que hacen el trabajo. La arquitectura que se repite en los proyectos públicos de este tipo es:

- **Transcripción con tiempos** (por ejemplo, con Whisper): convierte la voz en texto y marca en qué segundo se dijo cada frase.
- **Corte y unión** (FFmpeg): un programa de línea de comandos que recorta, une y convierte video.
- **Animaciones y rótulos** (Remotion): un sistema que genera video a partir de código.

Lo que el video no cuenta

- **El gusto se entrena.** Dice de pasada que tuvo que «iterar» la fábrica hasta que editara «con mi gusto». Ese gusto escrito (ritmo, tipo de animación, qué se corta) es la memoria de la fábrica de contenido. Sin esa guía de estilo, el resultado es genérico.
- **Solo es posproducción.** Lo que hace bueno a un video (tema, estructura, claridad) ocurre antes y sigue siendo humano.
- **Control de calidad.** Un corte automático puede quitar una frase que cambia el sentido. Por eso él revisa antes de subir, y hace bien.
- **Costo y derechos.** No habla del costo por video ni de las licencias de música e imágenes.

> **HECHO · LAS REGLAS DE YOUTUBE**
>
> YouTube no prohíbe el contenido hecho con IA. Su política de «contenido no auténtico» (nombre vigente desde julio de 2025, aclarada en julio de 2026) retira la monetización al contenido repetitivo, genérico o producido en masa, con o sin IA. Aparte, pide etiquetar el contenido realista alterado o sintético. Esto viene de prensa especializada; antes de publicar conviene leer el texto vigente en la ayuda oficial de YouTube.

> **PARA ARVYON**
>
> Tu canal encaja en el lado seguro: una persona real que explica, con edición asistida. El riesgo está en el extremo opuesto, los canales sin rostro que publican en serie. Cuando llegue la Fase 2, lo razonable es empezar por lo más barato y útil (recortar silencios, subtítulos, descripción y capítulos) y dejar las animaciones para después.

6 · *Harness engineering* y Pi

6.1 Qué es un arnés

Harness significa arnés. En IA, el arnés es **todo lo que hay en un agente salvo el modelo**: las instrucciones que recibe, la memoria, las herramientas que puede usar, los permisos y las comprobaciones que revisan su trabajo. La fórmula que circula es **agente = modelo + arnés**. Si el modelo es el motor, el arnés es el resto del coche: volante, frenos, tablero y cinturón.

Harness engineering (ingeniería del arnés) es el oficio de diseñar ese entorno para que el agente acierte a la primera y para que sus errores se corrijan antes de llegar a una persona. El término se popularizó en febrero de 2026 con el artículo de OpenAI, y Birgitta Böckeler, de Thoughtworks, lo ordenó en abril de 2026 con dos ideas:

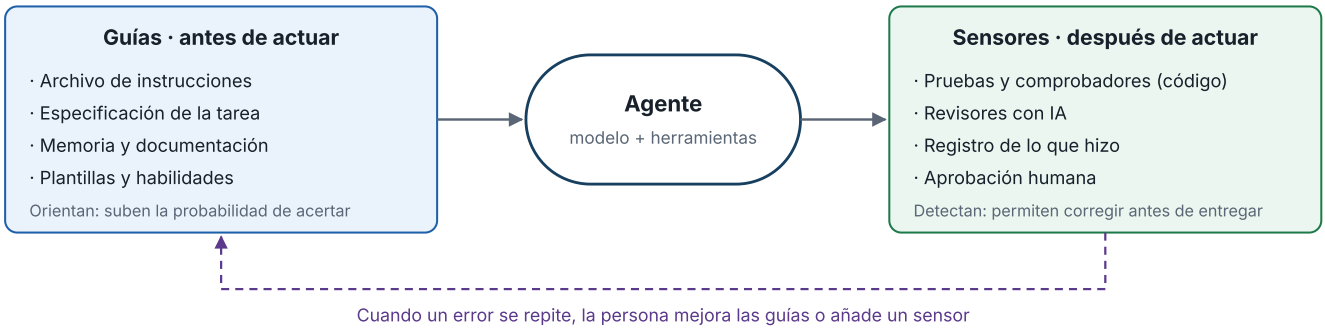


Figura 3. Guías y sensores, según Böckeler (martinfowler.com, 2026-04-02). Adaptación propia.

- **Guías y sensores.** Las guías actúan antes y orientan. Los sensores actúan después y detectan.
- **Computacional o inferencial.** Un control computacional lo ejecuta un programa: es rápido, barato y da siempre lo mismo. Uno inferencial lo ejecuta una IA: entiende el significado, pero es más lento, más caro y variable.

Böckeler reconoce lo que aún no está resuelto: comprobar el *comportamiento* (que el resultado hace lo que el negocio necesita) es «el elefante en la habitación», y no existe todavía una forma de medir si un arnés es bueno.

PARA ARVYON · YA TIENES UN ARNÉS

El cerebro que montamos es un arnés para un agente de conocimiento, y sus flujos son una pequeña fábrica:

Pieza del cerebro	Qué es en este vocabulario
`CLAUDE.md` y `PROMPT-MAESTRO.md`	Guías (instrucciones y reglas)
`index.md`, `log.md` y la wiki	Memoria, el *brain* del video
`herramientas/revisar_wiki.py`	Sensor computacional: siempre revisa lo mismo
Los recuadros «Por verificar» y «Contradicción»	Sensor inferencial: el agente juzga su propia evidencia
Tú uniendo cada *pull request*	Puerta humana
Flujo de ingesta en 12 pasos	Línea de fábrica: entra una fuente, salen páginas

6.2 Qué es Pi

Pi es un agente de programación de código abierto que se usa desde la terminal. Lo creó Mario Zechner a finales de 2025 porque Claude Code le parecía «una nave espacial» llena de funciones que no usaba. Desde abril de 2026 lo mantiene la empresa Earendil. Su lema: «Hay muchos arneses de agentes, pero este es tuyo».

Rasgo	Cómo es en Pi
Núcleo mínimo	Cuatro herramientas (leer, escribir, editar y ejecutar comandos) y unas instrucciones de sistema de menos de 1.000 *tokens*.
Lo que no trae a propósito	Subagentes, modo plan, ventanas de permiso, lista de tareas y procesos en segundo plano. El argumento: cada función que no usas ocupa contexto y esconde lo que hace el agente.
Cómo se amplía	Extensiones en TypeScript (pueden interceptar órdenes peligrosas, proteger archivos o añadir memoria), habilidades que se cargan solo cuando hacen falta, plantillas de instrucciones y paquetes instalables.
Modelos	Más de 15 proveedores (Anthropic, OpenAI, Google, modelos locales y otros). Se cambia de modelo y de nivel de razonamiento en la misma sesión.
Modos de uso	Interactivo, por órdenes para guiones, y como pieza dentro de otros programas.
Conectores (MCP)	Al principio no los traía. El 2026-11-11 su sitio dice que ya vienen integrados. Guías de septiembre de 2026 aún dicen lo contrario: es un cambio reciente.

6.3 Qué quiere decir «harness engineering basado en Pi»

Es usar Pi como base en blanco para construir tu propio arnés pieza a pieza, en vez de aceptar uno ya armado. Es el camino opuesto al del video: RSC Harness instala un arnés completo (memoria, habilidades, comprobaciones) encima de Claude Code o Codex; Pi te da el mínimo y tú añades solo lo que necesitas. Hay un curso abierto, «Learn Harness Engineering», con versión en español, 14 lecciones, 8 proyectos y un capítulo sobre el arnés de Pi. Exige manejar terminal, git y código.

RIESGOS QUE CONVIENE CONOCER

- **Pi no pide permiso antes de actuar.** Ejecuta con los permisos de tu usuario. Su autor considera que las ventanas de permiso de otros agentes son «teatro de seguridad» y recomienda aislarlo en un contenedor.
- **Los paquetes ejecutan código en tu máquina.** Hay que revisarlos antes de instalarlos.
- **La suscripción de Claude.** Según The Register (2026-02-20), Anthropic limita el acceso con suscripción personal a Claude Code y Claude.ai. Para usar Claude dentro de Pi, lo previsto es una clave de API de pago por uso.

PARA ARVYON

El concepto, sí; la herramienta, no por ahora. Saber qué es un arnés te sirve desde ya: explica por qué dos empresas con el mismo modelo obtienen resultados distintos, y eso es lo que vende un consultor. Pi es para quien programa a diario. Si después del 15 de diciembre quieres ver un arnés por dentro, ya tienes uno a mano y más sencillo: el propio cerebro.

7 · Enrutamiento de modelos: la disciplina que buscabas

Lo que describes (un sistema que decide qué modelo y cuánto razonamiento usar en cada situación, para gastar menos) se llama **enrutamiento de modelos**, en inglés *model routing* o *LLM routing*. La disciplina más amplia que lo contiene es **FinOps para IA**.

Nombre que puedes oír	Qué es
Enrutamiento de modelos	Enviar cada petición al modelo más barato que la resuelve bien. Es el nombre de la técnica.
Cascada de modelos	Probar primero el modelo barato y subir al caro solo si la respuesta no pasa una comprobación.
FinOps para IA	La gestión financiera del gasto en IA: ver cuánto se gasta, repartirlo por área y optimizarlo. La FinOps Foundation publica la guía de referencia.
Pasarela de IA (*AI gateway*)	El programa intermedio por donde pasan todas las llamadas. Ahí se aplican las rutas, los topes de gasto y el registro.
LLMOps	La operación de aplicaciones con modelos de lenguaje: evaluar, vigilar, versionar y controlar costos. El enrutamiento es una parte.
Nivel de razonamiento	El ajuste que indica al modelo cuánto «pensar» antes de responder. Más razonamiento da mejor resultado en lo difícil y gasta más.

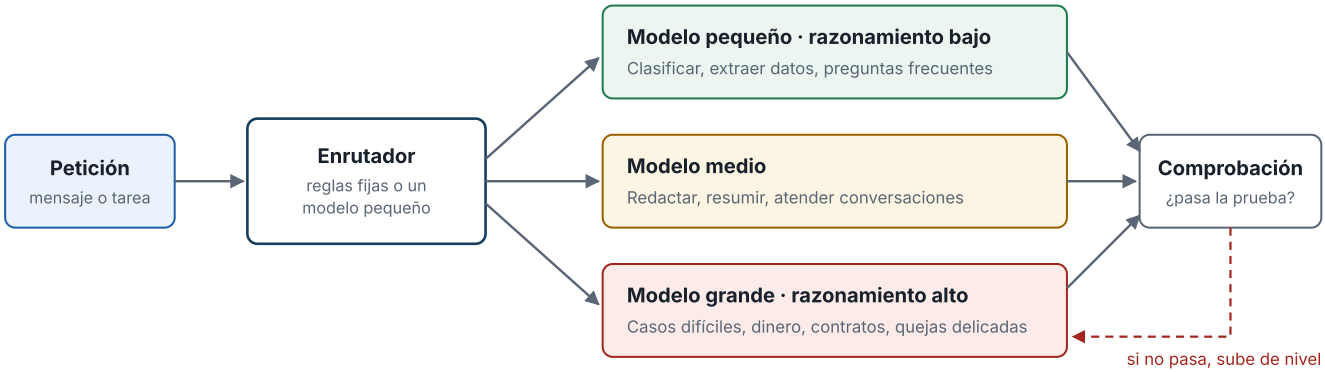


Figura 4. Enrutamiento con cascada. Los ejemplos de tareas por nivel son una propuesta mía para validar con pruebas, no una norma.

Las palancas para gastar menos

Palanca	Ahorro indicativo	En qué consiste
Elegir bien el modelo y enrutar	60–90 %	Lo descrito arriba. Un modelo puntero puede costar de 50 a 100 veces más por *token* que el más pequeño.
Caché de instrucciones	50–90 % de lo repetido	Lo que se envía igual en cada llamada (las instrucciones fijas) se cobra con descuento a partir de la segunda vez.
Proceso por lotes	cerca del 50 %	Para lo que no necesita respuesta inmediata, como un reporte nocturno.
Gestión del contexto	20–60 %	Enviar solo lo necesario: resumir el historial y recuperar el fragmento útil en vez del documento entero.
Limitar la respuesta	10–40 %	El texto que el modelo escribe suele costar de 3 a 5 veces más que el que lee.

Fuente: FinOps Foundation, «Managing AI Token Costs from Direct Model Provider APIs», actualizado el 2026-10-28. La propia guía presenta las cifras como indicativas y pide validarlas con datos propios. No da cifra para el nivel de razonamiento.

HECHO · LA MÉTRICA QUE IMPORTA

La guía recomienda medir el **costo por resultado exitoso**: lo que cuesta todo el proceso, contando solo los casos que superan el umbral de calidad. Los intentos fallidos también se pagan. Un modelo barato que falla la mitad de las veces puede salir más caro que uno bueno.

Lo que hay que saber antes de prometer ahorros

- **Sin pruebas no hay enrutamiento.** Para saber qué modelo «resuelve bien» una tarea hace falta un conjunto de casos de ejemplo con su respuesta correcta. Por eso las evaluaciones van primero.
- **Las cifras grandes son de laboratorio.** RouteLLM (2024) y FrugalGPT (2023) reportan ahorros muy altos en bancos de prueba. El resultado real depende de cuántas peticiones fáciles tenga cada empresa.
- **También existe la ruta inversa.** Anthropic ofrece una «estrategia del asesor»: un modelo económico hace el trabajo y consulta a uno potente solo cuando se atasca. Si consulta mucho, el ahorro desaparece.
- **Un buen arnés permite un modelo más barato.** Es el hallazgo de Jain del capítulo 4.
- **En una pyme, a menudo el modelo no es el costo mayor.** Suelen pesar más las horas de montaje, el mantenimiento y el tiempo de las personas que revisan.

PARA ARVYON · LAS CINCO PREGUNTAS DEL CONSULTOR

1. ¿Qué tareas hará la IA y cuántas veces al mes?
2. ¿Cuál es el modelo más barato que pasa la prueba en cada tarea?
3. ¿Qué parte de cada llamada se repite y se puede guardar en caché?
4. ¿Qué puede esperar y procesarse por lotes?
5. ¿Cuánto cuesta cada resultado correcto, comparado con hacerlo a mano?

En el video esta disciplina ya está aplicada sin nombrarla: el modelo más capaz construye y uno menor revisa el código.

8 · Temas que faltaban en tu lista

Al seguir el rastro de los cuatro temas aparecieron ocho más. Los ordeno por lo que pesan para una consultora de IA que atiende pymes.

#	Tema	Qué es y por qué te importa	Dónde encaja
1	**Evaluaciones (*evals*)**	Pruebas para una IA: se le da una entrada y se califica la salida. Son la base de todo lo anterior. Sin ellas no puedes saber si una automatización funciona, ni elegir modelo, ni demostrar resultados a un cliente.	Bloque 10 · semana 5
2	**Seguridad de agentes**	La **inyección de instrucciones**: un texto ajeno (un correo, un mensaje de WhatsApp, una página web) le da órdenes al agente. El riesgo es máximo cuando se juntan tres cosas, la «tríada letal» de Simon Willison: datos privados, contenido no confiable y un canal para enviar información fuera. La defensa es quitar una de las tres. OWASP publicó en diciembre de 2025 su lista de los diez riesgos de aplicaciones con agentes.	Bloque 8 · semanas 2 y 5
3	**Flujo o agente**	Anthropic distingue los **flujos** (el camino lo fija el código) de los **agentes** (el modelo decide su camino). La mayoría de los casos de una pyme son flujos, más baratos y predecibles. Saber decir «aquí no hace falta un agente» es criterio de consultor. Los cinco patrones: encadenar, enrutar, paralelizar, orquestador con trabajadores y evaluador con optimizador. La fábrica del video los usa casi todos.	Bloque 4 · semana 3
4	**Ingeniería de contexto**	El paso siguiente a escribir buenas instrucciones: decidir qué información llega al modelo en cada momento. El principio de Anthropic es el menor conjunto de información de alta señal que logre el resultado. Es lo que hace el *brain* del video.	Bloque 2 · semana 2
5	**Supervisión humana bien diseñada**	Decidir qué aprueba una persona, por qué canal y con qué información. n8n ya permite pausar a un agente y pedir aprobación por WhatsApp, Slack, Telegram o correo antes de ejecutar una acción.	Bloque 5 · semana 4 (traspaso a un humano)
6	**Escribir especificaciones**	La especificación del video es, para ti, el encargo bien escrito: qué se quiere, con qué criterios se acepta y cómo se comprueba. Es la habilidad que más se parece al trabajo diario de un consultor.	Bloque 11 · semana 5
7	**Registro y trazabilidad**	Que cada paso deje rastro: qué entró, qué salió, qué modelo se usó, cuánto costó. Es lo que el video llama «auditable» y la base de cualquier reporte al cliente.	Bloque 7 · semana 4
8	**Conectores y habilidades (MCP y *Skills*)**	Los dos estándares con que un agente se conecta a herramientas (MCP) y aprende procedimientos (habilidades). Basta con saber qué son y para qué sirven.	Bloque 6 · Fase 2

También apareció la norma ISO/IEC 42001 (2023), el primer estándar internacional de sistemas de gestión de IA. Pertenece al bloque 8 y se suma a lo que el cerebro ya tiene sobre la Ley 1581 y el Reglamento europeo de IA.

9 · Qué significa para Arvyon

9.1 La idea, traducida a una pyme

El patrón del video no es de programadores. Sirve para cualquier proceso repetible. Un ejemplo con una **ferretería ficticia** que responde cotizaciones por WhatsApp:

#	Paso	Tipo	Cómo se revisa
1	Llega un mensaje pidiendo una cotización	Entrada	Queda registrado con fecha y remitente
2	Extraer productos y cantidades del mensaje	Agente	Código: ¿están todos los campos?
3	Buscar precios y existencias en la hoja de inventario	Código	Siempre da lo mismo
4	Calcular total, IVA y descuento según la tabla	Código	Siempre da lo mismo
5	Redactar la cotización con el tono de la empresa	Agente	Código: ¿los números coinciden con el paso 4?
6	Regla fija: si supera un tope, pide un descuento fuera de tabla o falta un precio, va a una persona	Código	La regla está escrita, no la decide la IA
7	Aprobar o corregir	Humano	Queda registrado quién aprobó
8	Enviar y guardar en el registro	Código	Reporte semanal de casos y errores

Memoria de esta fábrica: lista de precios, política de descuentos, tono de la empresa y preguntas frecuentes. Es un ejemplo inventado para estudiar. Coincide con la demo 1 de tu semana 3.

9.2 Dónde encaja en tu plan, sin cambiar el calendario

Cuándo	Qué idea de este informe usar
Semanas 1 y 2	Agente = modelo + arnés. Ingeniería de contexto. Por qué existen modelos pequeños, medios y grandes.
Semana 3	Flujo o agente. Los tres tipos de pasos. Diseñar la demo de cotización como una fábrica.
Semana 4	Puerta humana por riesgo, no por tamaño. Registro de cada paso.
Semana 5	Evaluaciones, costo por resultado exitoso y las cinco preguntas del enrutamiento. La lista de doce piezas como guía de una propuesta.
Fase 2	Fábrica de contenido para el canal. Conectores y habilidades. Pi, si quieres ver un arnés por dentro.

9.3 Tres ejercicios con empresas ficticias

1. **Dibuja una fábrica.** Elige un proceso de una empresa inventada (por ejemplo, agendar citas en un consultorio) y llena una tabla como la de 9.1.
2. **Encuentra el riesgo escondido.** En esa fábrica, señala un cambio pequeño de alto riesgo, como el aviso legal del video, y escribe la regla que lo manda a una persona.

3. **Reparte los modelos.** Asigna a cada paso de agente un nivel (pequeño, medio o grande) y explica en una línea por qué.

9.4 Lo que no te recomiendo ahora

- Montar una fábrica de software o instalar RSC Harness o Pi. Requieren programar y no son lo que vende Arvyon.
- Esperar a RSC Factory. No estaba publicado el 2026-11-11.
- Cambiar el plan de estudio de esta semana. Estas ideas entran en bloques que ya están en el calendario.

9.5 Lo que el video enseña sobre YouTube

Con 424.685 vistas en 19 días, el video también es un caso de formato para tu canal:

- **Arranque con un dolor**, no con una presentación: «el cuello de botella eres tú».
- **Una analogía sencilla** antes de lo técnico: la fábrica de coches.
- **Un caso propio, paso a paso**, con un diagrama que se va completando.
- **Prueba social al final**: OpenAI y Stripe.
- **Giro para ampliar el público**: «esto también sirve a quien no programa».
- **Llamada a la acción ligada al siguiente video** y descripción con capítulos y enlaces.

9.6 Si lo apruebas, esto entraría al cerebro

- La transcripción en `raw/transcripciones/` y este informe en `raw/informes/`.
- Una página de fuente para el video y otra para el canal de Eric Risco, con las lecciones de formato.
- Tres conceptos nuevos: fábrica de IA, arnés de agentes y enrutamiento de modelos.
- Una página de análisis con la evaluación del video.
- Actualización del currículo (bloques 4, 6, 8 y 10), de tendencias y de lagunas.

10 · Glosario

Término	Significado
Agente	Un modelo de IA que usa herramientas y decide por sí mismo los pasos para cumplir una tarea.
Arnés (*harness*)	Todo lo que rodea al modelo en un agente: instrucciones, memoria, herramientas, permisos y comprobaciones.
Brain	Nombre que el video da a la memoria persistente del proyecto.
Builder	El agente constructor: el que escribe el código.
Caché de instrucciones	Descuento por reutilizar la parte del mensaje que no cambia entre llamadas.
CI (integración continua)	Sistema que ejecuta las comprobaciones automáticas cada vez que alguien propone un cambio.
Determinista	Que da siempre el mismo resultado con la misma entrada. El código lo es; un modelo de IA, no del todo.
Enrutamiento de modelos	Enviar cada petición al modelo más barato que la resuelve bien.
Especificación (*spec*)	Documento que dice qué hay que construir y con qué criterios se acepta.
Evaluación (*eval*)	Prueba para una IA: una entrada, una salida y una regla para calificarla.
Fábrica de IA	En el video: proceso repetible y auditable con IA dentro. Ojo: Nvidia usa el mismo nombre para centros de datos.
FinOps para IA	Gestión financiera del gasto en IA.
Inyección de instrucciones	Ataque en el que un texto ajeno le da órdenes al agente (*prompt injection*).
Issue	Aviso de error o petición de mejora en GitHub.
MCP	Estándar para conectar un agente con herramientas y datos externos.
Pi	Agente de programación minimalista y de código abierto, pensado para que cada quien construya su arnés.
***Pull request* (PR)**	Propuesta de cambio en GitHub que alguien debe aprobar antes de integrarla.
Sensor	Control que actúa después de que el agente trabaja y detecta errores.
Sentry	Servicio que registra los errores de una aplicación en funcionamiento.
Subagente	Agente que otro agente lanza para una tarea concreta, con su propio contexto.
TDD	Escribir primero la prueba y después el código que la supera.
Token	Unidad en que los modelos miden y cobran el texto. Aproximadamente, un trozo de palabra.
UX y UI	Experiencia de uso (si el recorrido se entiende) e interfaz (cómo se ve).

11 · Fuentes y límites de esta investigación

Fiabilidad de 1 a 5, con la escala del cerebro: 5 es fuente primaria revisada por pares u oficial; 1 es opinión sin respaldo.

Fuente	Fecha	Fiab.	Enlace
Eric Risco, «Programar con agentes ya no basta» (transcripción automática completa)	2026-10-23	3	youtube.com/watch?v=p6nnqerea7w
Eric Risco, repositorio RSC Harness	2026-11-11	3	github.com/ericrisco/rsc-harness
Stripe, «Minions: Stripe's one-shot, end-to-end coding agents»	2026-02-09	4	stripe.dev/blog/minions-stripes-one-shot-end-to-end-coding-agents
StrongDM, «Software Factory»	2026-02-06	3	factory.strongdm.ai
OpenAI, «Harness engineering» (leído a través de InfoQ)	2026-02-11	3	infoq.com/news/2026/02/openai-harness-engineering-codex
B. Böckeler, «Harness engineering», martinfowler.com	2026-04-02	4	martinfowler.com/articles/harness-engineering.html
N. Jain, «Are Harnesses Worth the Cost?», martinfowler.com	2026-10-13	4	martinfowler.com/articles/are-harnesses-worth-the-cost.html
FinOps Foundation, «Managing AI Token Costs from Direct Model Provider APIs»	2026-10-28	4	finops.org/wg/token-economics-saas/
Anthropic, «Building effective agents»	2024-12-19	4	anthropic.com/engineering/building-effective-agents
Anthropic, «Effective context engineering for AI agents»	2025	4	anthropic.com/engineering/effective-context-engineering-for-ai-agents
Anthropic, documentación de la herramienta «advisor»	2026	4	platform.claude.com/docs/en/agents-and-tools/tool-use/advisor-tool
Sitio oficial de Pi	2026-11-11	4	pi.dev
M. Zechner, «What I learned building an opinionated and minimal coding agent»	2025-11-30	4	mariozechner.at/posts/2025-11-30-pi-coding-agent/
Fazt, tutorial de Pi en español	2026-09-12	3	fazt.dev/contenido/pi-agent-tutorial-harness-skills-extensiones-mcp
walkinglabs, curso «Learn Harness Engineering» (versión en español)	2026	3	github.com/walkinglabs/learn-harness-engineering
Panickssery, Bowman y Feng, «LLM Evaluators Recognize and Favor Their Own Generations», NeurIPS 2024	2024	5	arxiv.org/abs/2404.13076
Ong y otros, «RouteLLM»; Chen, Zaharia y Zou, «FrugalGPT»	2024; 2023	4	arxiv.org/abs/2406.18665 · arxiv.org/abs/2305.05176
DORA, «State of AI-assisted Software Development 2025»	2025	4	services.google.com/fh/files/misc/2025_state_of_ai_assisted_software_development.pdf
METR, ensayo de productividad y actualización	2025-07; 2026-02-24	4	metr.org/blog/2026-02-24-uplift-update/

S. Willison, «Clinejection»; nota de Snyk	2026-03-06	4	simonwillison.net/2026/Mar/6/clinejection/
The Register, sobre el acceso con suscripción de Claude en herramientas de terceros	2026-02-20	3	theregister.com/software/2026/02/20/anthropic-clarifies-ban-on-third-party-tool-access-to-claude/5014546
TechCrunch, sobre la política de YouTube de contenido no auténtico	2026-07-20	3	techcrunch.com/2026/07/20/youtube-clarifies-policies-around-ai-slop-and-upsetting-videos/
n8n, documentación de aprobación humana para herramientas de agentes	2026	4	docs.n8n.io/advanced-ai/human-in-the-loop-tools
OpenAI, «GPT-6 Astra»; DeepLearning.AI, The Batch, sobre Sol	2026-09	3	openai.com/index/gpt-6-astra/ · deeplearning.ai/the-batch/sol-nearly-eclipses-astra
Repositorios de terceros basados en el video	2026-11	2	github.com/mieresantonio/fabrica-de-software · github.com/DOKK-droid/ai-factory

Límites

- **No vi la imagen del video.** Trabajé con los subtítulos automáticos, que tienen errores de transcripción. En el minuto 04:24 el autor da un porcentaje de mejora que el subtítulo no recoge.
- **No pude abrir la página original de OpenAI** (el sitio bloqueó la lectura automática). Sus cifras vienen de InfoQ y de resúmenes.
- **Varias cifras llegan por fuentes secundarias**: METR, DORA, OWASP, RouteLLM y FrugalGPT.
- **Las cifras de alcance y de estrellas** son del 2026-11-11 y cambian cada día.
- **No probé ninguna herramienta.** Es investigación de escritorio, como corresponde a la Fase 1.

POR VERIFICAR ANTES DE DARLO POR FIRME

- Las cifras exactas del experimento de OpenAI (número de cambios y tamaño del equipo) en su artículo original.
- Que Astra esté «entrenado con refuerzo» para revisar interfaces, como dice el video.
- El texto vigente de la política de YouTube y de las condiciones de Anthropic sobre suscripciones.
- Si RSC Factory ya se publicó.
- Los nombres exactos de las versiones de Sol: las fuentes no coinciden.